TASK 27

Reading and Writing Excel Data Using Python, Pandas & OpenPyXL

Internship track: Data Science & Data Analytics Using Python

Task objective: Read an Excel worksheet, inspect the data, clean common data-quality issues, perform a basic business analysis, and export processed data to Excel and CSV.

Project dataset: Synthetic employee data created for practice. It is not real employee or salary information.

Deliverables: source workbook, executable Python script, Jupyter Notebook, processed Excel workbook, processed CSV, README, requirements, execution notes, and this expanded guide.

1. Task overview and expected workflow

Excel is commonly used for operational reports, finance, human resources, inventory, and business analysis. Manual editing can become slow and inconsistent when datasets are large or tasks repeat. Python can automate the process and make it reproducible.

This task follows a standard data-processing workflow:

Stage	Purpose	Example in this project
1. Load	Import workbook data	Read the Employees worksheet
2. Inspect	Understand size, columns, types, and quality	Check shape, dtypes, missing values, duplicates
3. Clean	Resolve inconsistent formats and data issues	Trim spaces, convert salary/date, handle blanks
4. Transform	Create analysis-ready fields or structures	Convert salary to numeric; normalize text
5. Analyze	Summarize records into useful measures	Employee counts and salary statistics by department
6. Export	Save results for other users/tools	Excel workbook and CSV
7. Validate	Check that outputs are correct and complete	Compare row counts and inspect output sheets

2. Tools and installation

2.1 Python

Python is the programming language used to define the steps and automate the workflow. The solution is designed for Python 3.10 or newer.

2.2 Pandas

Pandas is the main data-analysis library. Its DataFrame structure represents data as rows and columns, similar to a worksheet or database table.

2.3 OpenPyXL

OpenPyXL is commonly used by Pandas to read and write modern .xlsx workbooks. It also allows formatting such as bold headers, column widths, filters, and frozen panes.

2.4 Jupyter Notebook

A notebook allows code to be executed cell by cell, with output and explanations beside the code. This is useful for learning and documenting analysis.

2.5 Setup

```
python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# macOS/Linux
source .venv/bin/activate

pip install -r requirements.txt
```

If the virtual environment activation command is blocked by local policy, you can still install packages in your chosen Python environment, or use the Python interpreter already configured in VS Code.

3. Understanding the project files

File	Role
employee_dataset.xlsx	Input workbook. Includes Employees data and a Data_Dictionary worksheet.
task27_excel_solution.py	Reusable script that reads, cleans, summarizes, and exports the data.
Task_27_Reading_Writing_Excel.ipynb	Interactive notebook with step-by-step code cells and learning notes.

File	Role
processed_employee_data.xlsx	Output workbook containing cleaned records, department summary, and data profile.
processed_employees.csv	Cleaned flat table in CSV format.
Task_27_Explanation.pdf	Original guide included in the starter package.
Task_27_Expanded_Explanation.pdf	Expanded guide with more concepts, examples, troubleshooting, and interview prep.
README.md	Project overview, setup, run instructions, and cleaning decisions.
requirements.txt	Python packages required to run the project.
execution_results.txt	Execution snapshot with row counts, missing-value checks, and department summary.

4. Reading Excel data with Pandas

```
import pandas as pd

df = pd.read_excel(
    "employee_dataset.xlsx",
    sheet_name="Employees",
    engine="openpyxl"
)
print(df.head())
```

pd.read_excel() loads data from a workbook into a DataFrame. The file path identifies the workbook; **sheet_name** selects the worksheet; **engine** explicitly identifies OpenPyXL as the .xlsx engine.

Useful options

• **sheet_name='Employees'** reads a sheet by name.

• **sheet_name=0** reads the first worksheet.

• **usecols=['Employee_ID', 'Salary']** loads only selected columns.

• **nrows=100** limits the read to the first 100 data rows.

• **header=0** means the first row contains column names.

If the file is not found, check that the script is running from the folder containing the workbook. The working directory may differ from the folder where the script is saved.

5. Inspecting structure and data quality

```
print("Shape:", df.shape)
print("Column names:", df.columns.tolist())
print("Data types:\n", df.dtypes)
print("Missing values:\n", df.isna().sum())
print("Duplicate rows:", df.duplicated().sum())
print(df.describe(include="all").transpose())
```

shape returns (row count, column count). **dtypes** shows how Pandas interprets each column. **isna().sum()** counts missing values by column. **duplicated().sum()** counts exact duplicate rows. **describe()** gives a quick statistical profile; the result varies by data type.

Inspection should happen before cleaning. For example, a salary column stored as text may look fine in Excel but cannot be averaged reliably until it is converted to numeric values.

6. Cleaning and transformation

6.1 Make a copy

```
cleaned = df.copy()
```

A separate copy preserves the original DataFrame for comparison and debugging. In real projects, keep the source workbook unchanged so transformations can be repeated and audited.

6.2 Normalize column labels and text

```
cleaned.columns = cleaned.columns.str.strip().str.replace(" ", "_")

for col in ["Employee_Name", "Department", "Location", "Email"]:
    cleaned[col] = cleaned[col].astype("string").str.strip()

cleaned["Employee_Name"] = cleaned["Employee_Name"].str.title()
cleaned["Location"] = cleaned["Location"].str.title()
```

strip() removes leading and trailing spaces. **title()** converts words to title case. This helps make values like ' noida ' and 'Noida' consistent. Do not apply title case to every field automatically: email addresses, IDs, and product codes may have different rules.

6.3 Handle missing department and email values

```
cleaned["Department"] = (
    cleaned["Department"]
    .replace({"": pd.NA, "nan": pd.NA, "None": pd.NA})
    .fillna("Unknown")
    .str.title()
)
cleaned["Email"] = cleaned["Email"].fillna("")
```

A missing department is replaced with 'Unknown' so that the record is not silently lost during grouping. Missing email values are converted to blank strings for convenient export. In other situations, retaining a true missing value may be more appropriate.

6.4 Convert salary strings to numeric

```
salary_text = (
    cleaned["Salary"]
    .astype("string")
    .str.replace(r"[■,$,]", "", regex=True)
)
cleaned["Salary"] = pd.to_numeric(salary_text, errors="coerce")
```

Currency symbols and comma separators are removed before conversion. **pd.to_numeric()** creates numeric values that can be added, averaged, sorted, or compared. With **errors='coerce'**, invalid values become missing values; always inspect them rather than assuming conversion succeeded.

6.5 Convert dates and ages

```
cleaned["Joining_Date"] = pd.to_datetime(
    cleaned["Joining_Date"], errors="coerce"
)
cleaned["Age"] = pd.to_numeric(cleaned["Age"], errors="coerce")
```

A datetime column supports date filtering, sorting, time intervals, and year/month analysis. Age must be numeric if it is to be averaged or checked against valid ranges.

6.6 Remove duplicate rows

```
cleaned = cleaned.drop_duplicates().reset_index(drop=True)
```

This removes rows that match across all columns. It does not guarantee that Employee_ID is unique if two rows have the same ID but different details. If Employee_ID is the business key, separately test for duplicate IDs and investigate conflicts before removing anything.

6.7 Fill missing age with the median

```
cleaned["Age"] = (
    cleaned["Age"]
    .fillna(cleaned["Age"].median())
    .round()
    .astype("Int64")
)
```

The median is the middle value after sorting and is less affected by extreme values than the mean. This is a simple practice choice—not a universal business rule. For a real dataset, consider leaving age missing, deriving it from a trusted date of birth, or using a documented imputation method.

6.8 Check the cleaned data

```
print("Rows after cleaning:", len(cleaned))
print("Missing values after cleaning:\n", cleaned.isna().sum())
print("Duplicate rows after cleaning:", cleaned.duplicated().sum())
```

```
print(cleaned.dtypes)
```

Always check after transformations. A conversion may create new missing values, or a cleaning operation may remove more records than expected.

7. Department-level analysis

```
summary = (
    cleaned.groupby("Department", dropna=False)
    .agg(
        Employee_Count=("Employee_ID", "count"),
        Average_Salary=("Salary", "mean"),
        Median_Salary=("Salary", "median"),
        Minimum_Salary=("Salary", "min"),
        Maximum_Salary=("Salary", "max"),
    )
    .reset_index()
    .sort_values("Employee_Count", ascending=False)
)
```

groupby() separates rows by department. **agg()** calculates several statistics for each group. Employee count describes group size; average and median salary describe central tendency; minimum and maximum show the range.

Do not interpret salary statistics without understanding the salary unit, whether values are monthly or annual, and whether all employee categories are comparable.

8. Exporting results to Excel and CSV

```
with pd.ExcelWriter(
    "processed_employee_data.xlsx",
    engine="openpyxl"
) as writer:
    cleaned.to_excel(
        writer, sheet_name="Cleaned_Employees", index=False
    )
    summary.to_excel(
        writer, sheet_name="Department_Summary", index=False
    )
    cleaned.describe(include="all").transpose().to_excel(
        writer, sheet_name="Data_Profile"
    )

cleaned.to_csv(
    "processed_employees.csv",
    index=False,
    encoding="utf-8-sig"
)
```

ExcelWriter makes it possible to save several tables as separate worksheets in one workbook. **index=False** prevents the DataFrame's row index from being added as an unwanted column. CSV stores one flat table and does not preserve multiple sheets, formulas, or cell formatting.

Optional workbook formatting

```
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill
from openpyxl.utils import get_column_letter

wb = load_workbook("processed_employee_data.xlsx")
for ws in wb.worksheets:
    ws.freeze_panes = "A2"
    ws.auto_filter.ref = ws.dimensions
    for cell in ws[1]:
        cell.font = Font(bold=True, color="FFFFFF")
        cell.fill = PatternFill("solid", fgColor="203864")
    for col in ws.columns:
        width = max(len(str(c.value or "")) for c in col) + 2
        ws.column_dimensions[get_column_letter(col[0].column)].width = min(width, 32)
wb.save("processed_employee_data.xlsx")
```

Formatting is for readability; it does not change the underlying data. Frozen panes keep headers visible while scrolling, filters help users narrow records, and column widths make values easier to read.

9. How to run the project

Option A: Run the Python script

```
# From the folder containing the script and input workbook:
python task27_excel_solution.py
```

The script prints raw shape, column names, missing-value counts, cleaned shape, and the department summary. It then creates the Excel and CSV outputs.

Option B: Run the Jupyter Notebook

```
pip install -r requirements.txt
jupyter notebook
```

Open **Task_27_Reading_Writing_Excel.ipynb**. Run cells from top to bottom. The notebook is useful when you want to see each stage and inspect intermediate DataFrames.

10. Results and validation

The supplied synthetic dataset begins with 41 rows and includes one exact duplicate row. After exact duplicate removal, the cleaned dataset contains 40 rows. The input workbook includes two sheets; the output workbook includes three analysis sheets.

Validation item	Expected check
Raw rows	41 records in the provided sample
Exact duplicates in source	1 exact duplicate row
Cleaned rows	40 records after exact duplicate removal
Salary type	Numeric values, with any unparseable entries flagged as missing
Joining_Date type	Datetime values
Output workbook	Cleaned_Employees, Department_Summary, Data_Profile
CSV output	One cleaned table; no worksheet formatting

Use these as validation targets for the supplied practice dataset. If you replace the dataset with your own file, the counts and summaries will naturally change.

11. Common errors and solutions

Problem	Why it happens	Solution
FileNotFoundError	The workbook is not in the current working directory, or the file name is wrong.	Check the folder, spelling, and file path; use Path.cwd() to inspect the working direc
Worksheet not found	sheet_name does not match a worksheet name.	Open the workbook and copy the exact sheet name.
Missing optional dependency openpyxl	The Excel engine is not installed in the active environment.	Run pip install openpyxl using the same Python environment.
Salary still appears as text	Symbols, spaces, or other unexpected characters remain.	Inspect distinct values, clean them, then use pd.to_numeric and check coercion-cr
Unnamed column in output	A prior export may have included the DataFrame index.	Use index=False and inspect the input workbook's first columns.
PermissionError when saving	The output workbook may be open in Excel or the folder may be restricted.	Close the workbook and save to a writable folder.
Notebook cannot import pandas	The notebook kernel uses a different Python environment.	Install requirements in the selected kernel environment and restart the kernel.

12. Interview / viva questions with answers

1. How do you read an Excel file using Pandas?

Use pd.read_excel('file.xlsx', sheet_name='Employees'). For .xlsx files, Pandas commonly uses OpenPyXL.

2. What is Pandas?

Pandas is a Python library for manipulating structured data using Series and DataFrame objects.

3. What is OpenPyXL?

A Python library for reading, writing, and formatting modern Excel .xlsx files.

4. What does df.shape return?

A tuple containing the number of rows and columns.

5. How do you inspect column data types?

Use df.dtypes.

6. How do you count missing values?

Use df.isna().sum().

7. How do you remove exact duplicate rows?

Use df.drop_duplicates(). Confirm the business definition of a duplicate first.

8. How do you convert a column to numeric?

Use pd.to_numeric(column, errors='coerce') after removing unwanted formatting characters.

9. What is the difference between read_excel and to_excel?

read_excel imports workbook data into Pandas; to_excel writes a DataFrame to a workbook.

10. Why use ExcelWriter?

It lets multiple DataFrames be written to separate sheets in one workbook.

11. Why use index=False?

It prevents the DataFrame index from becoming an extra output column.

12. What is the difference between Excel and CSV?

Excel supports multiple sheets, formulas, formatting, and workbook features; CSV is plain text for a single flat table.

13. Why use groupby?

To divide records into groups and calculate summaries for each group.

14. Why inspect after data cleaning?

To verify transformations worked, catch new missing values, and confirm row counts and types.

15. Is median imputation always correct?

No. It is a possible method, but the choice should be justified by data meaning and business rules.

GitHub submission details

Suggested commit summary

```
Task 27: read, clean, summarize, and export Excel data with Pandas
```

Suggested extended description

Completed Task 27 by implementing an Excel data-processing workflow with Python, Pandas, and OpenPyXL. The project reads a multi-sheet workbook, inspects structure and data quality, standardizes text fields, converts salary and date columns, handles missing values, removes exact duplicate records, generates a department-level summary, and exports cleaned results to Excel and CSV. Included a reusable script, notebook, synthetic dataset, requirements, README, output files, and expanded documentation.

Top 5 skills used

• Python Programming

• Pandas

• Excel Data Processing

• Data Cleaning and Preprocessing

• Data Analysis and Reporting

Sample feedback / learning reflection

This task gave me practical experience reading and writing Excel files using Python and Pandas. I learned to inspect dataset structure, identify missing and duplicate records, standardize text, convert numeric and date fields, summarize data by category, and export results for reporting. I also learned that cleaning decisions must be documented and validated against business rules.